Diapo 1 :
So let’s starts with a quick overview of DeltaConv

First of all, they propose 2 streams :
A scalar steam that process non directional information
A vector stream that process directional information

Those two streams are connected though geometric operator like gradient, co-gradient, divergence, curl, norm and also Hodge Laplacian fr vector to vect

Furthermore, multilayer perceptron introduce non linearity in both stream to combine features and enhance expressiveness.

A major point of the method is that it is coordinate independent, ensuring that the outputs do not depend on tangent basis choice, we will see that later.

The goals :
Capture anisotropic patterns on geometric data
And support diverse data (point clouds, images)

Diapo 2:

First of all this is a schematic overview of the convolutional layer.
We have a point cloud in 3D of N points, and a matrix of scalar features that associate C features to each points, each feature is encoded by a scalar, all these features are represented by the scalar stream

And the vector stream: the matrix of vector features of layer l associates C_l (which the features at layer l) to 2 coefficients, indeed each feature is encoded by two coefficients in the tangent plane.

Diapo 3:

So as we said Scalar stream represents scalar features and so non direction information
And vector stream represents vector feature, directional information.

Diapo 4 :

So now let’s see how we can pass from the scalar stream to the vector stream.
First of all, the gradient is use to transform a scalar field to a vectr field.
But DeltaConv operate on discrete point cloud, so first we need to approximate the gradients on discrete data using the moving least squares method.

We also construct the co-gradient by taking a 90 degree rotation of the gradient within the tangent plane.

The sparsity of the matrix , means that we take into account only its knearest neighbors.

Diapo 5:

Why also taking the cogradient ? It will span the full tangent plane and This ensures no directional information is missed, allowing the network to analyze scalar features in multiple directions (not just the steepest one)
And so enabling the network to
scale : adjust the magnitude of features along the gradient and co gradient directions
Skew : transform feature non-linearly within the tangent plane
Rotate: reorient features by mixing gradient and coggradient components

And so it captures all the directions.

Diapo 6:
A major issue with point clouds is that they often have regions that are undersampled or noisy. As we can see on the foot of the chair, we have some weird gradient.
For that they use a ridge regression/ This is another contribution of DeltaConv it's less prone to noisy data.

Another issue is that we could have by repeating the gradient operator in the network, exploding and vanishing gradients and so we need to normalise.

Diapo 7:
Now we need to pass from the vector stream, to the scalar stream. We have three geometric operators: divergence, curl and norm.
The divergence operator is constructed by the same way as gradient, and the curl operator is analogous to the co-gradient. The sparsity ensures that only the neighborhood of each point influences computation.

Diapo 8:
And so we can combine both. We can pass from scalar stream to vector stream with gradient, and back to scalar stream with divergence.
And by the same way we pass from vector stream to scalar stream with curl and back to scalar stream with the co-gradient.

Diapo 9:

Let's focus a bit more on the scalar stream. So first, for example we can have a plane as a point cloud and for a given point some features can be more important than others.

Diapo 10:
We first apply a classical MLP, weighted sum of the input layers, non linearity and repeat for a number of layers.

Diapo 11:
The activation function used in the MLP is Leaky RELU for the scalar stream.
WE could use the Laplacian

Diapo 12:
But instead we use a simplified version of EdgeConv and PointNet++,

Diapo 13:
And instead of using edge as explained by Julien, DeltaConv processes each point individually and learn features for each point individually but also captures importance features from its neighbors

Diapo 14:
So now let's focus a bit more on the vector stream. So as we've said it takes in input two coefficients in the tangent plane at a certain point.
It applies, like in scalar stream, a MLP in vectors , weighted sum of input feature, apply non linearity to norms and repeat. Here MLP use RELU as an activation function.

Diapo 15:
It applies Hodge Laplacian, which is a generalization of the Laplacian for tangent vector field.

Diapo 16:
And then We combine all the blocks we've seen. So it concatenates everything.

Diapo final:
The equation linearly combining the elementary oper-ators and composing them with non-linearities in-between to form
anisotropic geometric operators, and so DeltaConv can support anisotropy.

The operators we use are related to each other in a fundamental way. They form a metric version of the de Rham complex of a surface [Wardetzky 2006]. The following diagram lays out the connections described in the previous sections, where each of the operators maps between functions (scalar fields) and vector fields.
$X\ V\ X$
grad
div
curl
co-grad (5)
Note that the bottom row is a 90-degree rotated version of the top row. If we follow the diagram from left to right and apply grad and then curl to any function, the output will always be zero. The same holds for the path from right to left. The operators listed are first-order derivatives. Laplacians, which are second-order derivatives, can be formed by composing the first-order operators. For functions: to vector fields with grad and back again with div (Laplace-Beltrami). For vector fields: we go to scalars with div and curl and back again with grad and co-grad (Hodge-Laplacian). DeltaConv learns to combine these operators and supports anisotropy by adding non-linearities in-between

Handling Discrete Point Clouds
Instead of working on a continuous surface, DeltaConv operates on discrete point clouds. To approximate gradients on discrete data, it uses a moving least-squares (MLS) method.
Key Steps in MLS: (voir le cours)
1. Neighborhood Definition: For each point in the point cloud, define a k-nearest neighbors (k-nn) neighborhood.
2. Local Approximation: Fit a local polynomial function to the scalar features within the neighborhood. Minimize a weighted least-squares error to ensure the polynomial approximates the scalar field locally.
3. Gradient Computation: Compute the gradient of the locally fitted polynomial at the center point, estimating the gradient in the tangent Plane

rojection-based Methods.
Advantages
• Represent the shape as a whole (rather than partial views)
• Can reuse shape parametrization methods
• Enable the use of CNNs
Limitations
• Parametrizations are not unique
• Typically induce (often heavy) distortion
• Rarely used in practice anymor

View-based Methods.
Advantages
• Efficiency & simplicity
• Can use (pre-trained) CNNs!
• Can be used to optimize 3D shapes via differentiable rendering.
Limitations
• Are cumbersome for local analysis (e.g., segmentation)
• Are not adapted to deformable shapes
• Not great for topologically complex shapes

$$\frac{\partial y}{\partial t} = \nabla \cdot (c(|\nabla y|)\nabla y)$$